UNIVERSIDADE FEDERAL DO CEARÁ

Test Smells em projetos OpenSource

Disciplina: Qualidade de Software
Equipe: Pablo Vinicios da Silva Araújo - 574229 e Levi Gomes Ferreira - 556935
Projeto: https://github.com/camelot-dev/camelot
Fork:https://github.com/PabloVini28/camelot/tree/refatoracao-test-smells

Levi Gomes — 20 ocorrências

Tipos: Redundant Print (1) · Exception Handling (4) · Conditional Test Logic (3) · Verbose Test (2) · Unknown Test (10)

Redundant Print (1 ocorrência)

Registro 01/20

Eu estou atualmente trabalhando na refatoração do seguinte code smell: **Redundant Print** em `tests/test_errors.py`, `test_pdfium_backend_import_error`, linha 175 (dificuldade: fácil).
Minhas principais dificuldades na remoção do code smell são: • O print ficava dentro de um except que nunca executa (o pypdfium2 já está importado quando o teste roda), então ele nunca aparecia na saída. • O mesmo método também tem Exception Handling; a remoção foi feita isolada, antes da reescrita do try/except, para manter um commit por tipo.
Usei o CATSPY para: Definição do smell e exemplo de refatoração (remover o print e deixar a verificação só na asserção). Página: Redundant Print
Eu estou usando os seguintes métodos de refatoração para remover o code smell: • Remoção da instrução print(em); a verificação continua no assert.

Exception Handling (4 ocorrências)

Registro 02/20

Eu estou atualmente trabalhando na refatoração do seguinte code smell: **Exception Handling** em `tests/test_errors.py`, `test_pdfium_backend_import_error`, linha 167 (dificuldade: difícil).
Minhas principais dificuldades na remoção do code smell são: • O assert ficava só dentro do except, que nunca era executado: o teste passava sem verificar nada.

- Foi preciso entender por que a exceção não acontecia: o pypdfium2 é importado quando o camelot carrega, então o mock.patch.dict(sys.modules) não tem efeito.
- Usar só `pytest.raises`(match=...) criaria um Unknown Test novo, porque o TEMPY só reconhece linhas com assert; por isso a mensagem foi verificada com assert explícito.

Usei o CATSPY para:
Definição e refatoração sugerida (trocar o try/except manual pelo recurso de exceção do framework: assertRaises no unittest, `pytest.raises` no pytest). Página: Exception Handling

Eu estou usando os seguintes métodos de refatoração para remover o code smell:
- Simulação da falha de importação com monkeypatch em `PDFIUM_EXC`.
- `pytest.raises`(OSError) as excinfo chamando `PdfiumBackend()`.convert diretamente, seguido de assert sobre a mensagem.

Registro 03/20

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Exception Handling em `tests/test_errors.py`, `test_no_tables_found_logs_suppressed`, linha 50 (dificuldade: fácil).

Minhas principais dificuldades na remoção do code smell são:
- O try/except capturava o warning só para chamar `pytest.fail`, o que repete o que o warnings.simplefilter("error") já faz.

Usei o CATSPY para:
Definição e refatoração sugerida (trocar o try/except manual pelo recurso de exceção do framework: assertRaises no unittest, `pytest.raises` no pytest). Página: Exception Handling

Eu estou usando os seguintes métodos de refatoração para remover o code smell:
- Remoção do try/except + `pytest.fail`; qualquer warning vira erro e falha o teste sozinho.

Registro 04/20

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Exception Handling em `tests/test_errors.py`, `test_no_tables_found_warnings_suppressed`, linha 62 (dificuldade: fácil).

Minhas principais dificuldades na remoção do code smell são:
- Mesmo padrão do teste anterior, com o arquivo empty.pdf.

Usei o CATSPY para:
Definição e refatoração sugerida (trocar o try/except manual pelo recurso de exceção do framework: assertRaises no unittest, `pytest.raises` no pytest). Página: Exception Handling

Eu estou usando os seguintes métodos de refatoração para remover o code smell:
- Remoção do try/except + `pytest.fail`, mantendo o simplefilter("error").

Registro 05/20

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Exception Handling em `tests/test_cli.py`, `test_cli_quiet`, linha 288 (dificuldade: fácil).

Minhas principais dificuldades na remoção do code smell são:
- O simplefilter("error") estava comentado, então o except Warning nunca disparava e o bloco era inútil.

Usei o CATSPY para:
Definição e refatoração sugerida (trocar o try/except manual pelo recurso de exceção do framework: assertRaises no unittest, `pytest.raises` no pytest). Página: Exception Handling

Eu estou usando os seguintes métodos de refatoração para remover o code smell:
- Remoção do try/except em volta do runner.invoke.

Conditional Test Logic (3 ocorrências)

Registro 06/20

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Conditional Test Logic em `tests/test_cli.py`, `test_help_output`, linha 27 (dificuldade: fácil).

Minhas principais dificuldades na remoção do code smell são:
- O for estava dentro de um all(...); o TEMPY detecta a linha que começa com for, mesmo sendo um gerador.

Usei o CATSPY para:
Definição e refatoração (extrair cada caso da condicional ou do laço para um teste próprio; no pytest isso é feito com parametrize). Página: Conditional Test Logic

Eu estou usando os seguintes métodos de refatoração para remover o code smell:
- A verificação das opções do --help foi para o novo teste `test_help_output_lists` com @`pytest.mark.parametrize`, um caso por opção.

Registro 07/20

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Conditional Test Logic em `tests/test_vector_engine_probe.py`, `test_lattice_engine_validation`, linha 62 (dificuldade: média).

Minhas principais dificuldades na remoção do code smell são: • O método misturava dois laços com objetivos opostos (valores aceitos e valores rejeitados); foi preciso separar em testes com nomes próprios.
Usei o CATSPY para: Definição e refatoração (extrair cada caso da condicional ou do laço para um teste próprio; no pytest isso é feito com parametrize). Página: Conditional Test Logic
Eu estou usando os seguintes métodos de refatoração para remover o code smell: • Laço dos valores válidos substituído por @`pytest.mark.parametrize` no teste `test_lattice_engine_accepts_valid_values`.

Registro 08/20

Eu estou atualmente trabalhando na refatoração do seguinte code smell: **Conditional Test Logic** em `tests/test_vector_engine_probe.py`, `test_lattice_engine_validation`, linha 65 (dificuldade: média).
Minhas principais dificuldades na remoção do code smell são: • O caso inválido, separado com só `pytest.raises`(match=...), viraria um Unknown Test novo para o TEMPY; a verificação da mensagem já foi feita com assert explícito.
Usei o CATSPY para: Definição e refatoração (extrair cada caso da condicional ou do laço para um teste próprio; no pytest isso é feito com parametrize). Página: Conditional Test Logic
Eu estou usando os seguintes métodos de refatoração para remover o code smell: • Laço dos valores inválidos substituído pelo teste parametrizado `test_lattice_engine_rejects_invalid_values`, com assert sobre excinfo.

Verbose Test (2 ocorrências)

Registro 09/20

Eu estou atualmente trabalhando na refatoração do seguinte code smell: **Verbose Test** em `tests/test_cli.py`, `test_cli_stream`, linha 60–113 (dificuldade: difícil).
Minhas principais dificuldades na remoção do code smell são: • O teste verificava cinco cenários diferentes da CLI em sequência dentro de um TemporaryDirectory; separar exigiu decidir quais dados cada cenário compartilha. • Uma linha longa já existente passou a ser reformatada pelo ruff format quando a indentação mudou.
Usei o CATSPY para: Definição (método longo com várias responsabilidades) e refatoração (dividir em métodos menores e focados). Página: Verbose Test

Eu estou usando os seguintes métodos de refatoração para remover o code smell:
- Divisão em 5 testes (saída csv, --output obrigatório, formato inferido pela extensão, --margins válido e inválido).
- Fixture budget_paths com tmp_path no lugar do TemporaryDirectory.

Registro 10/20

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Verbose Test em `tests/test_cli.py`, `test_cli_output_format`, linha 225–277 (dificuldade: média).

Minhas principais dificuldades na remoção do code smell são:
- Quatro blocos quase idênticos (json, excel, html, markdown) e um caso diferente (--zip) no mesmo método.

Usei o CATSPY para:
Definição (método longo com várias responsabilidades) e refatoração (dividir em métodos menores e focados). Página: Verbose Test

Eu estou usando os seguintes métodos de refatoração para remover o code smell:
- @`pytest.mark.parametrize` para os quatro formatos.
- Teste separado `test_cli_output_format_zip` para o --zip.

Unknown Test (10 ocorrências)

Registro 11/20

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Unknown Test em `tests/test_errors.py`, `test_unknown_flavor`, linha 15 (dificuldade: fácil).

Minhas principais dificuldades na remoção do code smell são:
- O teste já verificava a mensagem com `pytest.raises`(..., match=...), mas o TEMPY só reconhece linhas com assert, então para a ferramenta ele não tinha asserção.

Usei o CATSPY para:
Definição (teste sem asserção clara passa sempre que nenhuma exceção ocorre) e refatoração (introduzir asserções que validem o resultado esperado). Página: Unknown Test

Eu estou usando os seguintes métodos de refatoração para remover o code smell:
- `pytest.raises`(...) as excinfo seguido de assert <mensagem> in str(excinfo.value), tornando a verificação explícita.

Registro 12/20

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

Unknown Test em `tests/test_errors.py`, `test_input_kwargs`, linha 24 (dificuldade: fácil).
Minhas principais dificuldades na remoção do code smell são: • O teste já verificava a mensagem com `pytest.raises`(..., match=...), mas o TEMPY só reconhece linhas com assert, então para a ferramenta ele não tinha asserção.
Usei o CATSPY para: Definição (teste sem asserção clara passa sempre que nenhuma exceção ocorre) e refatoração (introduzir asserções que validem o resultado esperado). Página: Unknown Test
Eu estou usando os seguintes métodos de refatoração para remover o code smell: • `pytest.raises`(...) as excinfo seguido de assert <mensagem> in str(excinfo.value), tornando a verificação explícita.

Registro 13/20

Eu estou atualmente trabalhando na refatoração do seguinte code smell: **Unknown Test** em `tests/test_errors.py`, `test_no_tables_found_logs_suppressed`, linha 45 (dificuldade: fácil).
Minhas principais dificuldades na remoção do code smell são: • Depois da remoção do try/except, o teste só verificava que nenhum warning era emitido; faltava conferir o resultado da leitura.
Usei o CATSPY para: Definição (teste sem asserção clara passa sempre que nenhuma exceção ocorre) e refatoração (introduzir asserções que validem o resultado esperado). Página: Unknown Test
Eu estou usando os seguintes métodos de refatoração para remover o code smell: • O retorno de read_pdf passou a ser verificado: assert len(tables) == 1 para foo.pdf.

Registro 14/20

Eu estou atualmente trabalhando na refatoração do seguinte code smell: **Unknown Test** em `tests/test_errors.py`, `test_no_tables_found_warnings_suppressed`, linha 57 (dificuldade: fácil).
Minhas principais dificuldades na remoção do code smell são: • Mesmo caso do anterior, com um PDF sem tabelas.
Usei o CATSPY para: Definição (teste sem asserção clara passa sempre que nenhuma exceção ocorre) e refatoração (introduzir asserções que validem o resultado esperado). Página: Unknown Test
Eu estou usando os seguintes métodos de refatoração para remover o code smell: • assert len(tables) == 0 para empty.pdf.



UNIVERSIDADE FEDERAL DO CEARÁ

Registro 15/20

Eu estou atualmente trabalhando na refatoração do seguinte code smell: **Unknown Test** em `tests/test_errors.py`, `test_no_password`, linha 69 (dificuldade: fácil).
Minhas principais dificuldades na remoção do code smell são: • O teste já verificava a mensagem com `pytest.raises`(..., match=...), mas o TEMPY só reconhece linhas com assert, então para a ferramenta ele não tinha asserção.
Usei o CATSPY para: Definição (teste sem asserção clara passa sempre que nenhuma exceção ocorre) e refatoração (introduzir asserções que validem o resultado esperado). Página: Unknown Test
Eu estou usando os seguintes métodos de refatoração para remover o code smell: • `pytest.raises`(...) as excinfo seguido de assert <mensagem> in str(excinfo.value), tornando a verificação explícita.

Registro 16/20

Eu estou atualmente trabalhando na refatoração do seguinte code smell: **Unknown Test** em `tests/test_errors.py`, `test_bad_password`, linha 76 (dificuldade: fácil).
Minhas principais dificuldades na remoção do code smell são: • O teste já verificava a mensagem com `pytest.raises`(..., match=...), mas o TEMPY só reconhece linhas com assert, então para a ferramenta ele não tinha asserção.
Usei o CATSPY para: Definição (teste sem asserção clara passa sempre que nenhuma exceção ocorre) e refatoração (introduzir asserções que validem o resultado esperado). Página: Unknown Test
Eu estou usando os seguintes métodos de refatoração para remover o code smell: • `pytest.raises`(...) as excinfo seguido de assert <mensagem> in str(excinfo.value), tornando a verificação explícita.

Registro 17/20

Eu estou atualmente trabalhando na refatoração do seguinte code smell: **Unknown Test** em `tests/test_errors.py`, `test_stream_equal_length`, linha 83 (dificuldade: fácil).
Minhas principais dificuldades na remoção do code smell são: • O teste já verificava a mensagem com `pytest.raises`(..., match=...), mas o TEMPY só reconhece linhas com assert, então para a ferramenta ele não tinha asserção.
Usei o CATSPY para:

Definição (teste sem asserção clara passa sempre que nenhuma exceção ocorre) e refatoração (introduzir asserções que validem o resultado esperado). Página: Unknown Test
Eu estou usando os seguintes métodos de refatoração para remover o code smell: • `pytest.raises`(...) as excinfo seguido de assert <mensagem> in str(excinfo.value), tornando a verificação explícita.

Registro 18/20

Eu estou atualmente trabalhando na refatoração do seguinte code smell: **Unknown Test** em `tests/test_errors.py`, `test_lattice_unknown_backend`, linha 133 (dificuldade: fácil).
Minhas principais dificuldades na remoção do code smell são: • O teste já verificava a mensagem com `pytest.raises`(..., match=...), mas o TEMPY só reconhece linhas com assert, então para a ferramenta ele não tinha asserção.
Usei o CATSPY para: Definição (teste sem asserção clara passa sempre que nenhuma exceção ocorre) e refatoração (introduzir asserções que validem o resultado esperado). Página: Unknown Test
Eu estou usando os seguintes métodos de refatoração para remover o code smell: • `pytest.raises`(...) as excinfo seguido de assert <mensagem> in str(excinfo.value), tornando a verificação explícita.

Registro 19/20

Eu estou atualmente trabalhando na refatoração do seguinte code smell: **Unknown Test** em `tests/test_errors.py`, `test_extraction_not_allowed`, linha 206 (dificuldade: fácil).
Minhas principais dificuldades na remoção do code smell são: • O teste só verificava o tipo da exceção, sem nenhum assert.
Usei o CATSPY para: Definição (teste sem asserção clara passa sempre que nenhuma exceção ocorre) e refatoração (introduzir asserções que validem o resultado esperado). Página: Unknown Test
Eu estou usando os seguintes métodos de refatoração para remover o code smell: • `pytest.raises`(PDFTextExtractionNotAllowed) as excinfo e assert "Text extraction is not allowed" in str(excinfo.value).

Registro 20/20

Eu estou atualmente trabalhando na refatoração do seguinte code smell: **Unknown Test** em `tests/test_cli.py`, `test_cli_quiet`, linha 280 (dificuldade: difícil).



UNIVERSIDADE
FEDERAL DO CEARÁ

Minhas principais dificuldades na remoção do code smell são:

- O teste nunca testou nada: a opção --quiet do camelot recebe um valor (is_flag=False), e passada sem valor ela engolia o "--format"; o comando terminava com erro de uso (exit code 2) que ninguém verificava.

Usei o CATSPY para:
Definição (teste sem asserção clara passa sempre que nenhuma exceção ocorre) e refatoração (introduzir asserções que validem o resultado esperado). Página: Unknown Test

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

- Argumentos corrigidos para --quiet true.
- Warnings registrados com catch_warnings(record=True).
- Asserts de exit_code == 0 e de nenhum warning emitido.



UNIVERSIDADE FEDERAL DO CEARÁ

Pablo Vinicios — 20 ocorrências

Tipos: Non-Functional Statement (1) · Exception Handling (3) · Conditional Test Logic (4) · Programming Paradigms Blend (2) · Unknown Test (10)

Non-Functional Statement (1 ocorrência)

Registro 01/20

Eu estou atualmente trabalhando na refatoração do seguinte code smell: **Non-Functional Statement** em `tests/test_errors.py`, `test_lattice_no_convert_method`, linha 143 (dificuldade: fácil).
Minhas principais dificuldades na remoção do code smell são: • O pass era o corpo de uma classe que propositalmente não tem o método `convert()`; não dava para só apagar sem deixar a classe vazia (erro de sintaxe).
Usei o CATSPY para: Definição e refatoração (remover a instrução sem efeito ou substituir por algo com finalidade). Página: Non-Functional Statement
Eu estou usando os seguintes métodos de refatoração para remover o code smell: • O pass virou uma docstring que explica a intenção. • A classe foi renomeada para BackendWithoutConvert.

Exception Handling (3 ocorrências)

Registro 02/20

Eu estou atualmente trabalhando na refatoração do seguinte code smell: **Exception Handling** em `tests/test_cli.py`, `test_import_error`, linha 327 (dificuldade: difícil).
Minhas principais dificuldades na remoção do code smell são: • O assert ficava dentro de um except ImportError que nunca executa, porque camelot.cli já está em cache quando o teste roda. • Recarregar o módulo com importlib.reload recria os objetos do Click e quebra outros testes de `test_cli`.py; foi preciso isolar a importação.
Usei o CATSPY para: Definição e refatoração sugerida (trocar o try/except manual pelo recurso de exceção do framework: assertRaises no unittest, `pytest.raises` no pytest). Página: Exception Handling
Eu estou usando os seguintes métodos de refatoração para remover o code smell: • Import de uma cópia nova de camelot.cli com o matplotlib oculto em sys.modules. • Módulo original restaurado com monkeypatch; assert `_HAS_MPL` is False.



UNIVERSIDADE FEDERAL DO CEARÁ

Registro 03/20

Eu estou atualmente trabalhando na refatoração do seguinte code smell: **Exception Handling** em `tests/test_export.py`, `test_export_sqlite`, linha 56 (dificuldade: média).
Minhas principais dificuldades na remoção do code smell são: • O TEMPY marca try/finally (fechamento da conexão SQLite), embora o catálogo descreva o smell com try/except; foi preciso achar o equivalente idiomático.
Usei o CATSPY para: Definição e refatoração sugerida (trocar o try/except manual pelo recurso de exceção do framework: assertRaises no unittest, `pytest.raises` no pytest). Página: Exception Handling
Eu estou usando os seguintes métodos de refatoração para remover o code smell: • try/finally `conn.close()` substituído por with contextlib.closing(sqlite3.connect(...)).

Registro 04/20

Eu estou atualmente trabalhando na refatoração do seguinte code smell: **Exception Handling** em `tests/test_export.py`, `test_table_to_sqlite`, linha 92 (dificuldade: média).
Minhas principais dificuldades na remoção do code smell são: • Mesmo padrão do teste anterior; a conexão precisava continuar sendo fechada mesmo se a consulta falhasse.
Usei o CATSPY para: Definição e refatoração sugerida (trocar o try/except manual pelo recurso de exceção do framework: assertRaises no unittest, `pytest.raises` no pytest). Página: Exception Handling
Eu estou usando os seguintes métodos de refatoração para remover o code smell: • with contextlib.closing(...) no lugar do try/finally.

Conditional Test Logic (4 ocorrências)

Registro 05/20

Eu estou atualmente trabalhando na refatoração do seguinte code smell: **Conditional Test Logic** em `tests/test_common.py`, `test_overlapping_text_preserves_adjacent_cell`, linha 480 (dificuldade: difícil).
Minhas principais dificuldades na remoção do code smell são: • O laço percorria as linhas de um DataFrame e fazia um assert por linha; foi preciso uma forma vetorizada com pandas que mantivesse a mensagem de erro útil (quais linhas falharam).

Usei o CATSPY para: Definição e refatoração (extrair cada caso da condicional ou do laço para um teste próprio; no pytest isso é feito com parametrize). Página: Conditional Test Logic
Eu estou usando os seguintes métodos de refatoração para remover o code smell: • agg(" ".join, axis=1) + str.contains(regex) para achar as linhas sem o número. • Uma única asserção sobre o conjunto de linhas faltantes.

Registro 06/20

Eu estou atualmente trabalhando na refatoração do seguinte code smell: **Conditional Test Logic** em `tests/test_export.py`, `test_export_sqlite`, linha 59 (dificuldade: média).
Minhas principais dificuldades na remoção do code smell são: • O for apontado era a segunda linha de uma list comprehension quebrada em várias linhas; reformatar não seria uma refatoração de verdade. • O any("page-1" in n ...) aceitava qualquer nome parecido, então valia fortalecer a verificação.
Usei o CATSPY para: Definição e refatoração (extrair cada caso da condicional ou do laço para um teste próprio; no pytest isso é feito com parametrize). Página: Conditional Test Logic
Eu estou usando os seguintes métodos de refatoração para remover o code smell: • Consulta com ORDER BY e `fetchall()`. • Comparação por igualdade com a lista exata de tabelas esperadas.

Registro 07/20

Eu estou atualmente trabalhando na refatoração do seguinte code smell: **Conditional Test Logic** em `tests/test_utils.py`, `test_bbox_from_str_rejects_zero_area`, linha 215 (dificuldade: fácil).
Minhas principais dificuldades na remoção do code smell são: • Laço simples sobre duas entradas inválidas.
Usei o CATSPY para: Definição e refatoração (extrair cada caso da condicional ou do laço para um teste próprio; no pytest isso é feito com parametrize). Página: Conditional Test Logic
Eu estou usando os seguintes métodos de refatoração para remover o code smell: • @`pytest.mark.parametrize` com as duas entradas.

Registro 08/20

Eu estou atualmente trabalhando na refatoração do seguinte code smell: **Conditional Test Logic** em `tests/test_ml_backend.py`, `test_ml_ocr_requires_engine_when_missing`, linha 195 (dificuldade: fácil).

Minhas principais dificuldades na remoção do code smell são: • O if dentro do teste decidia em tempo de execução se ele devia ser pulado.
Usei o CATSPY para: Definição e refatoração (extrair cada caso da condicional ou do laço para um teste próprio; no pytest isso é feito com parametrize). Página: Conditional Test Logic
Eu estou usando os seguintes métodos de refatoração para remover o code smell: • if + `pytest.skip()` substituído por @`pytest.mark.skipif`.

Programming Paradigms Blend (2 ocorrências)

Registro 09/20

Eu estou atualmente trabalhando na refatoração do seguinte code smell: **Programming Paradigms Blend** em `tests/test_image_conversion_backend.py`, `(nível do módulo)`, linha 6; 19 (dificuldade: média).
Minhas principais dificuldades na remoção do code smell são: • As classes apontadas não são classes de teste, e sim backends falsos misturados com fixture e funções de teste no mesmo arquivo. • Foi preciso escolher entre transformar os testes em classes ou tirar as classes do arquivo; mantive o estilo do projeto (funções de teste).
Usei o CATSPY para: Definição e refatoração (não misturar código procedural e orientado a objetos no mesmo arquivo de teste). Página: Programming Paradigms Blend
Eu estou usando os seguintes métodos de refatoração para remover o code smell: • Criação do módulo `tests/doubles.py` com os quatro backends falsos. • O arquivo de teste passou a importá-los e ficou só com funções.

Registro 10/20

Eu estou atualmente trabalhando na refatoração do seguinte code smell: **Programming Paradigms Blend** em `tests/test_utils.py`, `(nível do módulo)`, linha 16; 115 (dificuldade: média).
Minhas principais dificuldades na remoção do code smell são: • A classe _TextlineStub ficava no meio de funções utilitárias e testes; foi preciso conferir que nenhum outro arquivo dependia dela antes de mover.
Usei o CATSPY para: Definição e refatoração (não misturar código procedural e orientado a objetos no mesmo arquivo de teste). Página: Programming Paradigms Blend
Eu estou usando os seguintes métodos de refatoração para remover o code smell: • Classe movida para `tests/doubles.py` como TextlineStub.

- O helper _make_textline continua no arquivo de teste usando a classe importada.

Unknown Test (10 ocorrências)

Registro 11/20

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Unknown Test em `tests/test_per_page.py`, `test_per_page_invalid_kwarg_rejected`, linha 45 (dificuldade: fácil).

Minhas principais dificuldades na remoção do code smell são:
- O teste já verificava a mensagem com `pytest.raises`(..., match=...), mas o TEMPY só reconhece linhas com assert, então para a ferramenta ele não tinha asserção.

Usei o CATSPY para:
Definição (teste sem asserção clara passa sempre que nenhuma exceção ocorre) e refatoração (introduzir asserções que validem o resultado esperado). Página: Unknown Test

Eu estou usando os seguintes métodos de refatoração para remover o code smell:
- `pytest.raises`(...) as excinfo seguido de assert <mensagem> in str(excinfo.value), tornando a verificação explícita.

Registro 12/20

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Unknown Test em `tests/test_per_page.py`, `test_per_page_invalid_flavor_rejected`, linha 55 (dificuldade: fácil).

Minhas principais dificuldades na remoção do code smell são:
- O teste já verificava a mensagem com `pytest.raises`(..., match=...), mas o TEMPY só reconhece linhas com assert, então para a ferramenta ele não tinha asserção.

Usei o CATSPY para:
Definição (teste sem asserção clara passa sempre que nenhuma exceção ocorre) e refatoração (introduzir asserções que validem o resultado esperado). Página: Unknown Test

Eu estou usando os seguintes métodos de refatoração para remover o code smell:
- `pytest.raises`(...) as excinfo seguido de assert <mensagem> in str(excinfo.value), tornando a verificação explícita.

Registro 13/20

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Unknown Test em `tests/test_per_page.py`, `test_per_page_auto_flavor_rejected`, linha 60 (dificuldade: fácil).

Minhas principais dificuldades na remoção do code smell são:
- O teste já verificava a mensagem com `pytest.raises(..., match=...)`, mas o TEMPY só reconhece linhas com assert, então para a ferramenta ele não tinha asserção.

Usei o CATSPY para:
Definição (teste sem asserção clara passa sempre que nenhuma exceção ocorre) e refatoração (introduzir asserções que validem o resultado esperado). Página: Unknown Test

Eu estou usando os seguintes métodos de refatoração para remover o code smell:
- `pytest.raises(...)` as excinfo seguido de assert <mensagem> in str(excinfo.value), tornando a verificação explícita.

Registro 14/20

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Unknown Test em `tests/test_per_page.py`, `test_per_page_non_dict_value_rejected`, linha 66 (dificuldade: fácil).

Minhas principais dificuldades na remoção do code smell são:
- O teste já verificava a mensagem com `pytest.raises(..., match=...)`, mas o TEMPY só reconhece linhas com assert, então para a ferramenta ele não tinha asserção.

Usei o CATSPY para:
Definição (teste sem asserção clara passa sempre que nenhuma exceção ocorre) e refatoração (introduzir asserções que validem o resultado esperado). Página: Unknown Test

Eu estou usando os seguintes métodos de refatoração para remover o code smell:
- `pytest.raises(...)` as excinfo seguido de assert <mensagem> in str(excinfo.value), tornando a verificação explícita.

Registro 15/20

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Unknown Test em `tests/test_utils.py`, `test_bbox_from_str_rejects_zero_area`, linha 212 (dificuldade: fácil).

Minhas principais dificuldades na remoção do code smell são:
- O teste já verificava a mensagem com `pytest.raises(..., match=...)`, mas o TEMPY só reconhece linhas com assert, então para a ferramenta ele não tinha asserção.
- Depende da parametrização feita no commit de Conditional Test Logic.

Usei o CATSPY para:
Definição (teste sem asserção clara passa sempre que nenhuma exceção ocorre) e refatoração (introduzir asserções que validem o resultado esperado). Página: Unknown Test

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

- `pytest.raises`(...) as excinfo seguido de assert <mensagem> in str(excinfo.value), tornando a verificação explícita.

Registro 16/20

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Unknown Test em `tests/test_utils.py`, `test_bbox_from_str_rejects_malformed`, linha 220 (dificuldade: fácil).

Minhas principais dificuldades na remoção do code smell são:
- O teste tinha dois `pytest.raises` seguidos; cada um precisou da sua própria variável para não confundir as verificações.

Usei o CATSPY para:
Definição (teste sem asserção clara passa sempre que nenhuma exceção ocorre) e refatoração (introduzir asserções que validem o resultado esperado). Página: Unknown Test

Eu estou usando os seguintes métodos de refatoração para remover o code smell:
- `pytest.raises`(...) as too_few e as not_numbers, cada um com seu assert.

Registro 17/20

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Unknown Test em `tests/test_utils.py`, `test_image_bbox_to_pdf_rejects_degenerate_input`, linha 276 (dificuldade: fácil).

Minhas principais dificuldades na remoção do code smell são:
- O teste já verificava a mensagem com `pytest.raises`(..., match=...), mas o TEMPY só reconhece linhas com assert, então para a ferramenta ele não tinha asserção.

Usei o CATSPY para:
Definição (teste sem asserção clara passa sempre que nenhuma exceção ocorre) e refatoração (introduzir asserções que validem o resultado esperado). Página: Unknown Test

Eu estou usando os seguintes métodos de refatoração para remover o code smell:
- `pytest.raises`(...) as excinfo seguido de assert <mensagem> in str(excinfo.value), tornando a verificação explícita.

Registro 18/20

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Unknown Test em `tests/test_ml_backend.py`, `test_ml_without_deps_raises_helpful_error`, linha 142 (dificuldade: fácil).

Minhas principais dificuldades na remoção do code smell são:
- O match usava regex com colchetes escapados (camelot-py\[ml\]); na asserção explícita virou comparação literal.

Usei o CATSPY para: Definição (teste sem asserção clara passa sempre que nenhuma exceção ocorre) e refatoração (introduzir asserções que validem o resultado esperado). Página: Unknown Test
Eu estou usando os seguintes métodos de refatoração para remover o code smell: • assert "camelot-py[ml]" in str(excinfo.value).

Registro 19/20

Eu estou atualmente trabalhando na refatoração do seguinte code smell: **Unknown Test** em `tests/test_ml_backend.py`, `test_ml_read_pdf_surfaces_install_hint`, linha 149 (dificuldade: fácil).
Minhas principais dificuldades na remoção do code smell são: • Mesmo caso do anterior, passando pelo read_pdf completo.
Usei o CATSPY para: Definição (teste sem asserção clara passa sempre que nenhuma exceção ocorre) e refatoração (introduzir asserções que validem o resultado esperado). Página: Unknown Test
Eu estou usando os seguintes métodos de refatoração para remover o code smell: • assert "camelot-py[ml]" in str(excinfo.value).

Registro 20/20

Eu estou atualmente trabalhando na refatoração do seguinte code smell: **Unknown Test** em `tests/test_ml_backend.py`, `test_ml_ocr_requires_engine_when_missing`, linha 193 (dificuldade: fácil).
Minhas principais dificuldades na remoção do code smell são: • Depende do skipif feito no commit de Conditional Test Logic; a regex do [ocr] virou comparação literal.
Usei o CATSPY para: Definição (teste sem asserção clara passa sempre que nenhuma exceção ocorre) e refatoração (introduzir asserções que validem o resultado esperado). Página: Unknown Test
Eu estou usando os seguintes métodos de refatoração para remover o code smell: • assert "camelot-py[ocr]" in str(excinfo.value).